CS6886 Assignment 5: Optimising Attention on a CPU

Manoj SS (BE22B028)
Repository: https://github.com/mangnit/cs6886_a5

November 6, 2026

Setup

- **Compiler:** clang 23.1.4 with the Makefile's flags, `-O3 -march=native` (plus `-fopenmp` for `omp`). The only Makefile change is the path in `CC`.
- **Timing:** the driver reports the median of 5 timed calls after 1 warm up call. Every (variant, input, threads) combination was run in 5 interleaved rounds, so slow drift affects every variant alike, and the fastest of the 5 medians is reported. Under WSL2 the spread between rounds was 7 to 40% for single threaded runs and up to 116% at 5 to 8 threads, so differences below about 10% are not significant.
- **FLOP count:** GFLOP/s uses the driver's count of $4N^2d$ FLOPs ($2N^2d$ for $QK^\top$ plus $2N^2d$ for SV; softmax is not counted).
- **Correctness:** `make check` passes for every variant on every input and for every OpenMP thread count. The largest error is 6.2×10^{-6}, against the 10^{-4} limit.

Q1. Machine

Field	Value
CPU model	Intel Core i5-1135G7 (11th Gen, Tiger Lake), Dell Inspiron 5502
Physical / logical cores	4 / 8 (2 way Hyper-Threading)
Performance / efficiency cores	4 P cores / 0 E cores (not hybrid)
Base clock	2.4 GHz
Max single core clock	4.2 GHz (measured: 4.12 GHz)
All core clock under load	2.8 GHz measured with all 8 threads busy (3.43 GHz with 4). The nominal 3.8 GHz all core turbo is not sustained by this laptop's power limit.
SIMD	AVX-512 (16 FP32 lanes); also AVX2 and FMA3 (8 lanes)
FMA units per core	2 × 256 bit, which act as 1 × 512 bit unit for AVX-512: 32 FP32 FLOP/cycle either way
L1d / L2 / L3	48 KB per core / 1.25 MB per core / 8 MB shared by all 4 cores
RAM	8 GB DDR4-3200, **1 channel** (single DIMM)
OS	Windows 11 with WSL2 (Ubuntu 20.04)

Table 1: Machine specification (full version in `machine_spec.txt`).

Peak FP32 throughput. Each core can start one 512 bit FMA per cycle (or two 256 bit FMAs, the same throughput), and an FMA counts as 2 FLOPs:

$$\text{FLOP/cycle/core} = 16 \text{ lanes} \times 2 \ \tfrac{\text{FLOP}}{\text{FMA}} \times 1 \ \tfrac{\text{FMA}}{\text{cycle}} = 32$$

- **One core** at the 4.2 GHz single core turbo: $32 \times 4.2 = \mathbf{134.4}$ GFLOP/s.

- **All cores** at the measured all core clock of 2.8 GHz: $4 \times 32 \times 2.8 = \mathbf{358.4}$ GFLOP/s. (At the nominal 3.8 GHz it would be 486.4, but the laptop does not sustain that clock.) Hyper-Threading adds no FMA units, so 8 threads have the same peak as 4 cores.

Memory bandwidth.

$$BW = 3200 \text{ MT/s} \times 8 \text{ B per transfer} \times 1 \text{ channel} = \mathbf{25.6\ GB/s}$$

Ridge points.

$$\text{one core: } \frac{134.4}{25.6} = \mathbf{5.25} \text{ FLOP/B} \qquad \text{all cores: } \frac{358.4}{25.6} = \mathbf{14.0} \text{ FLOP/B}$$

Measured check. I measured the clock and FMA throughput with my own microbenchmark, and the bandwidth with a STREAM triad:

threads	clock (GHz)	FMA peak (GFLOP/s)	FLOP/cycle per busy core	triad (GB/s)
1	4.12	129.7	31.5	12.8
2	3.97	245.9	31.0	13.5
4	3.43	310.4	22.6	14.4
8	2.80	353.5	31.6	15.2

Table 2: Measured clock, FMA throughput and triad bandwidth.

One core reaches 96% of its peak (31.5 FLOP/cycle), which confirms 32 FLOP/cycle. All cores reach 353.5 of 358.4 GFLOP/s. The triad reaches 15.2 GB/s (20.2 GB/s counting the write allocate read), about 79% of 25.6 GB/s, which is normal for single channel DDR4. A read sweep over working set sizes gives about 100 GB/s from L1, 50 to 60 GB/s from L2 and about 22 GB/s from L3 for one thread; these are used below.

Q2. Naive baseline

N	d	time (ms)	GFLOP/s	% of 1 core peak (134.4)
1024	64	104.19	2.58	1.9%
2048	64	431.96	2.49	1.8%
4096	64	1919.62	2.24	1.7%
4096	128	6602.31	1.30	1.0%

Table 3: `make naive` on every input.

Time against N. The work grows as $4N^2d$, so doubling N should multiply the time by 4. Measured: 1024 to 2048 is $\times$4.15 and 2048 to 4096 is $\times$4.44. GFLOP/s drops slightly with N because more data falls out of cache: at $N = 1024$ the matrix S is 4 MB and fits in the 8 MB L3, while at $N = 2048$ and 4096 it is 16 MB and 64 MB and lives in DRAM.

Time against d. Doubling d (4096 $\times$ 64 to 4096 $\times$ 128) doubles the FLOPs but multiplies the time by **3.44**. Almost all of the extra cost is in SV, which takes 4.9$\times$ longer for 2$\times$ the work (Table 4).

Where the time goes. I timed each step separately with a small harness that includes the variant's source file and puts timers between the same loops (a separate run, so totals differ a little from the tables). The table also covers the Q3 variants.

variant	total (ms)	pack	$QK^\top$	softmax: max	softmax: exp + sum	softmax: normalise	SV	$QK^\top$ GFLOP/s	SV GFLOP/s
$N = 4096$, $d = 64$									
naive	1987.6	0.0 (0%)	706.5 (36%)	19.3 (1%)	64.9 (3%)	3.0 (0%)	1186.1 (60%)	3.0	1.8
loops	325.2	0.9 (0%)	115.7 (36%)	21.1 (6%)	70.9 (22%)	3.3 (1%)	110.4 (34%)	18.6	19.4
tiled	174.0	0.2 (0%)	46.0 (26%)	21.6 (12%)	73.1 (42%)	3.4 (2%)	26.5 (15%)	46.7	81.0
fused	380.2	0.9 (0%)	112.3 (30%)	22.9 (6%)	81.1 (21%)	0.2 (0%)	161.3 (42%)	19.1	13.3
best	63.2	0.5 (1%)	25.8 (41%)	0.0 (0%)	8.6 (14%)	0.1 (0%)	26.0 (41%)	83.2	82.7
$N = 4096$, $d = 128$									
naive	7396.3	0.0 (0%)	1523.8 (21%)	19.3 (0%)	64.7 (1%)	3.0 (0%)	5794.3 (78%)	2.8	0.7
loops	677.0	2.1 (0%)	281.8 (42%)	23.5 (3%)	79.6 (12%)	3.7 (1%)	281.2 (42%)	15.2	15.3
tiled	248.2	0.5 (0%)	75.3 (30%)	23.9 (10%)	81.0 (33%)	3.7 (1%)	60.0 (24%)	57.0	71.6
fused	637.3	1.8 (0%)	237.9 (37%)	23.3 (4%)	82.6 (13%)	0.2 (0%)	290.9 (46%)	18.1	14.8
best	111.6	1.1 (1%)	49.1 (44%)	0.0 (0%)	8.1 (7%)	0.2 (0%)	50.2 (45%)	87.4	85.6

Table 4: Time per step in ms (share of the total). In `best` the max is part of the online softmax and is counted under exp + sum.

1. $QK^\top$ **(36%): a chain of dependent scalar FMAs.** The inner loop `acc += Q[i][k] * K[j][k]` sums into one scalar. Without `-ffast-math`, C's floating point rules forbid reordering that sum, so clang cannot vectorise it (*"loop not vectorized: cannot prove it is safe to reorder floating-point operations"*). Each scalar FMA waits 4 cycles for the previous one, which allows about 0.5 FLOP/cycle, or 2 GFLOP/s. It measures 3.0 GFLOP/s because the out of order core overlaps the chain of the next j; that is still only about 2% of peak.
2. SV **(60% at $d = 64$, 78% at $d = 128$): the same scalar chain, plus a walk down a column of V.** Consecutive iterations read `V[j][k]` and `V[j+1][k]`, which are $4d$ bytes apart, so every load touches a different cache line. The next 15 values of k use the same lines again, but only if the 4096 lines touched by one pass over j are still in cache.
   - **At $d = 64$ they are.** L2 has 1024 sets of 20 ways. A 256 B stride reaches only every 4th set, so the 4096 lines share 256 sets, about 16 lines per set, which fits. The loop runs close to the FMA latency limit: 1.8 GFLOP/s.
   - **At $d = 128$ they are not.** A 512 B stride reaches only every 8th set: about 32 lines per set, more than the 20 ways. Each line is evicted before $k + 1$ needs it, so every access fetches a whole 64 B line from L3. At 22 GB/s that is 64 B per 2 FLOPs, about 0.7 GFLOP/s, exactly what is measured. This is the 4.9× slowdown for 2× the work.
3. **Softmax (4%):** $N^2 = 16.8$ M scalar `expf` calls take about 65 ms (about 16 cycles each), and the max search is another dependent scalar chain. This cost does not depend on d.
4. **Storing S costs little here.** At $N = 4096$, S is 64 MB and goes to DRAM, about 340 MB of traffic (see Q5). At the measured 15 GB/s that is about 23 ms, about 1% of the run. The naive code is limited by instruction latency and cache misses inside the core, not by DRAM.

Q3. Optimisations

variant	N	d	time (ms)	GFLOP/s	% of 1 core peak	speedup	`make check` (error)
loops	1024	64	16.21	16.56	12.3%	6.4×	PASS (0.0)
loops	2048	64	68.46	15.68	11.7%	6.3×	PASS (0.0)
loops	4096	64	306.27	14.02	10.4%	6.3×	PASS (0.0)
loops	4096	128	537.81	15.97	11.9%	12.3×	PASS (1.8×10^{-6})
tiled	1024	64	8.04	33.40	24.8%	13.0×	PASS (0.0)
tiled	2048	64	34.54	31.09	23.1%	12.5×	PASS (0.0)
tiled	4096	64	155.58	27.61	20.5%	12.3×	PASS (0.0)
tiled	4096	128	209.37	41.03	30.5%	31.5×	PASS (0.0)
fused	1024	64	17.64	15.22	11.3%	5.9×	PASS (2.2×10^{-6})
fused	2048	64	69.97	15.34	11.4%	6.2×	PASS (3.2×10^{-6})
fused	4096	64	303.06	14.17	10.5%	6.3×	PASS (6.2×10^{-6})
fused	4096	128	584.20	14.70	10.9%	11.3×	PASS (4.9×10^{-6})
best	1024	64	3.32	80.78	60.1%	31.4×	PASS (2.5×10^{-6})
best	2048	64	12.92	83.09	61.8%	33.4×	PASS (3.5×10^{-6})
best	4096	64	50.40	85.22	63.4%	**38.1**×	PASS (6.1×10^{-6})
best	4096	128	98.56	87.16	64.8%	**67.0**×	PASS (5.7×10^{-6})

Table 5: Every single threaded variant on every input; speedup is over naive. Plots of every input are in `plots/`.

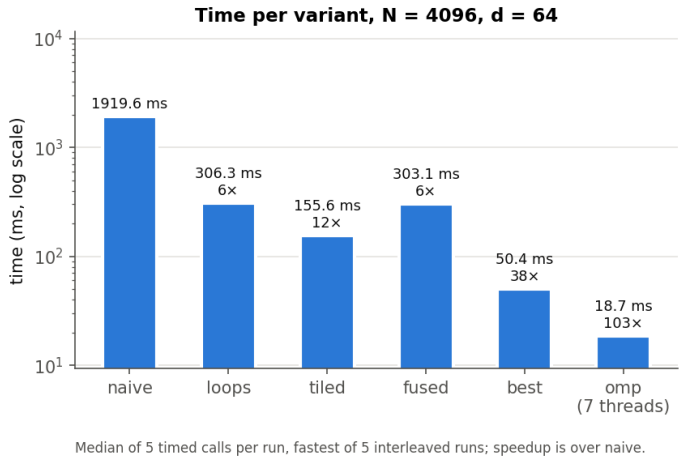


Figure 1: Time of every variant at $N = 4096$, $d = 64$ (`plots/time_N4096_d64.png`).

Figure 2 shows how the variants move the data: (a) `naive`, `loops` and `tiled` store the whole $N \times N$ matrix S; (b) `fused` keeps one row of it but reads all of K and V again for every row; (c) `best` works on blocks, so every K or V block loaded into L1 is used by 32 rows.

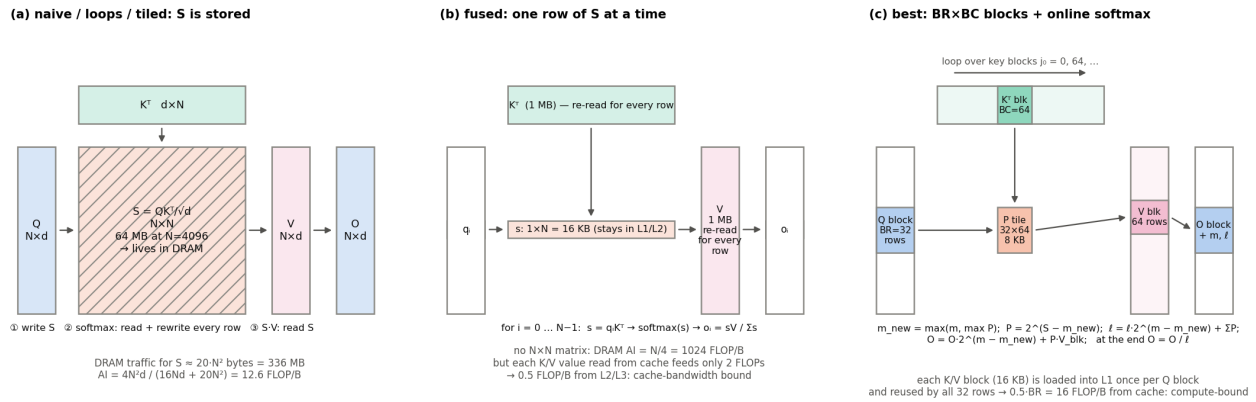


Figure 2: How each family of variants moves the data.

(a) `attn_loops.c`: loop reordering, 6.3× (12.3× at $d = 128$)

What changed. K is transposed once into $K^\top$ (about 1 ms). $QK^\top$ now runs in i, k, j order, so the inner loop is `S[i][j] += Q[i][k] * Kt[k][j]`: unit stride along j, and each $S[i][j]$ is a separate element, so there is no reduction. SV runs in i, j, k order, so the inner loop `O[i][k] += S[i][j] * V[j][k]` walks along a row of V.

Effect. clang now vectorises both inner loops (*"vectorized loop (vectorization width: 8, interleaved count: 4)"*), and the matmuls go from 1.8 to 3.0 GFLOP/s up to 18.6 to 19.4 GFLOP/s. Each output element is still summed in the same order, so at $d = 64$ the result is bit for bit identical to naive. This also needs the scale $1/\sqrt{64} = 1/8$ to be a power of two, since `loops` scales Q before the sum and naive scales the sum after. At $d = 128$ it is not, hence the 1.8×10^{-6} error. The speedup is larger at $d = 128$ because naive gets much slower there (Q2) while the softmax, which does not depend on d, is a smaller share of `loops`' time.

What limits it afterwards.

- **No register reuse:** every FMA loads an element of $K^\top$ (or V) and loads and stores an element of the S (or O) row. $K^\top$ (1 MB) is streamed from L2 again for every row, 4 bytes per 2 FLOPs, so the 50 to 60 GB/s from L2 allows about 25 to 30 GFLOP/s; 19 is measured.
- **The scalar softmax**, now 29% of the time. `expf` is a libm call that may set `errno`, so clang cannot vectorise it, and the max loop is a reduction.

(b) `attn_tiled.c`: cache and register tiling, 12.3× (31.5× at $d = 128$)

What changed.

- $QK^\top$**:** $K^\top$ is packed into contiguous panels of 32 columns (8 KB). Each 4×32 tile of S is accumulated in 16 vector registers over the whole k loop, so every loaded $K^\top$ vector is used by 4 rows and every Q value by 32 columns: 8 loads per 16 FMAs, against 2 loads and 1 store per FMA in (a). A panel stays in L1 while a block of 128 rows of Q passes over it.
- SV**:** 4×32 register tiles of O are accumulated over blocks of 256 rows of V (64 KB), which stay in L2 while all rows of the block use them.
- **Tile sizes** came from a sweep: 4×32 and 6×32 were equal; 8×32 and 4×64 were 9% slower because their 32 accumulators no longer fit in the 32 vector registers and spill to the stack.

Effect. $QK^\top$ goes from 18.6 to 46.7 GFLOP/s and SV from 19.4 to 81 GFLOP/s. $QK^\top$ is the slower of the two because it also writes the 64 MB S to DRAM, and each store into a new line first reads it (write allocate).

What limits it afterwards. The scalar softmax, now 56% of the time (98 of 174 ms), and the stored S, which still goes through DRAM. Since the softmax does not depend on d, `tiled` reaches 41 GFLOP/s at $d = 128$ but only 27.6 at $d = 64$.

(c) `attn_fused.c`: row by row fusion, 6.3× (11.3× at $d = 128$)

What changed. For each row i the three steps run back to back on one row of scores (N floats, 16 KB): $s = q_i K^\top$, then max, exp and sum, then $O_i = (\sum_j s_j V_j)/\text{sum}$. The division is moved to the d outputs. The $N \times N$ matrix is never stored: scratch memory is 16 KB instead of 64 MB at $N = 4096$.

Effect: about the same speed as `loops` (303 against 306 ms).

- The DRAM traffic removed (about 336 MB for S) cost `loops` only about 25 ms of 306, so it was never the bottleneck.
- Each row now reads all of $K^\top$ and V, 2 MB together, which is more than the 1.25 MB L2. So SV reads V partly from L3 and takes 161 ms against 110 ms in `loops`.

What limits it afterwards. Cache bandwidth: each K or V element brought in from cache is used for only 2 FLOPs (0.5 FLOP/B), and 22 to 55 GB/s (L3 to L2) allows only about 11 to 28 GFLOP/s; it gets 14. The scalar softmax is another 27%. Fusion only pays off when several rows share each K and V block, which is what (d) does.

(d) `attn_best.c`: blocked, fused attention with AVX-512 (FlashAttention style)

38× at $N = 4096$, $d = 64$ (85.2 GFLOP/s, 63% of the single core peak) and 67× at $d = 128$ (87.2 GFLOP/s, 65%).

What I did and why it works.

1. **Blocking with an online softmax.** Query rows are processed in blocks of $BR = 32$ and keys in blocks of $BC = 64$. Each row keeps a running max m and sum ℓ. For each key block:

$$m_{\text{new}} = \max(m, \max_j s_j), \qquad p_j = 2^{\,s_j - m_{\text{new}}},$$
$$\ell \leftarrow \ell \cdot 2^{\,m - m_{\text{new}}} + \textstyle\sum_j p_j, \qquad O \leftarrow O \cdot 2^{\,m - m_{\text{new}}} + P\,V_{\text{block}}$$

   and at the end $O \leftarrow O/\ell$. This is exactly the softmax, computed in pieces. S is never stored, and each K and V block (16 KB) is loaded into L1 once and used by all 32 rows, so the FLOPs per byte read from L2 or L3 rise from 0.5 (`fused`) to $0.5 \cdot 32 = 16$.
2. **Register tiled AVX-512 kernels.** $QK^\top$ and PV both use 4×64 tiles (16 zmm accumulators). Each step issues 4 vector loads and 4 broadcasts for 16 FMAs, so the FMA unit, not the load ports, is the limit. Each matmul runs at 83 to 99 GFLOP/s.
3. **Vectorised exp.** e^x is computed as $2^{x \log_2 e}$, with $\log_2 e/\sqrt{d}$ folded into Q when it is packed. $2^x = 2^n \cdot 2^f$ with $n = \text{round}(x)$: 2^f is a degree 6 polynomial (relative error below 2×10^{-7}) and 2^n comes from `vscalefps`. This replaces about 16 cycles per scalar `expf` with about 12 vector instructions per 16 values: the softmax takes 8.6 ms instead of 98 ms in `tiled`.
4. **Packing with padding.** Q, $K^\top$ (in panels of 64 keys) and V are copied once into aligned buffers, with N padded to the block size and d to a multiple of 16, so the kernels need no tail loops. Padded keys get a score of $-\infty$ and zero rows of V, so they add nothing. Packing takes 0.5 ms.
5. **Block sizes from a sweep:** $BR = 32$ to 64 was best ($BR = 4$ was 20% slower), and $BC = 64$ to 256 were equal. I chose $BR = 32$, $BC = 64$, the smallest working set among the best.

What limits it now. The time is now $QK^\top$ 41%, PV 41%, softmax 14%, packing 1%. The limit is the FMA unit itself: the matmuls reach 64 to 76% of the measured single core FMA peak (129.7 GFLOP/s), and the exp polynomial and rescaling need the same unit. The rest is overhead from the per row max and sum reductions, the small score tile and loop control.

Q4. OpenMP (`attn_omp.c`)

What changed. `attn_omp.c` is `attn_best.c` with the packing loop and the loop over query row blocks shared among threads. Row blocks are independent, so each thread has its own score tile, accumulator and running max and sum, and the threads share only the read only packed Q, K, V. The row block loop uses `schedule(dynamic, 1)`: with `static`, 5 threads were slower than 4 (1.60 against 0.99 ms at $N = 1024$), because two threads share a core and everyone waits for that slow pair.

The thread count was set with `OMP_NUM_THREADS`. Speedup $= T_1/T_p$ and efficiency $= T_1/(pT_p)$, where T_1 is `attn_omp` with 1 thread.

	$N = 1024$, $d = 64$			$N = 2048$, $d = 64$			$N = 4096$, $d = 64$			$N = 4096$, $d = 128$		
threads	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
1	3.18	1.00×	100%	12.36	1.00×	100%	56.34	1.00×	100%	99.40	1.00×	100%
2	1.68	1.89×	95%	9.35	1.32×	66%	34.89	1.61×	81%	59.10	1.68×	84%
3	1.27	2.50×	83%	5.54	2.23×	74%	23.48	2.40×	80%	51.69	1.92×	64%
4	**1.00**	3.18×	79%	**4.55**	2.72×	68%	20.08	2.81×	70%	46.48	2.14×	53%
5	1.07	2.98×	60%	4.88	2.53×	51%	23.88	2.36×	47%	41.81	2.38×	48%
6	1.11	2.88×	48%	4.84	2.55×	43%	20.85	2.70×	45%	39.86	2.49×	42%
7	1.21	2.64×	38%	4.80	2.58×	37%	**18.72**	3.01×	43%	38.17	2.60×	37%
8	1.29	2.46×	31%	4.68	2.64×	33%	20.28	2.78×	35%	**37.35**	2.66×	33%

Table 6: `attn_omp`: time, speedup over 1 thread and parallel efficiency (bold: fastest). `make check` passes for every row.

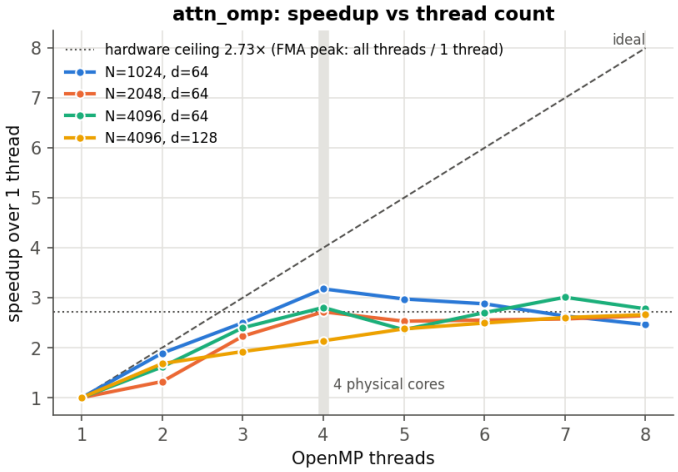


Figure 3: Speedup over 1 thread. The dotted line is the 8 thread hardware ceiling.

Explaining the scaling. The speedup is 1.3 to 1.9× with 2 threads and 2.1 to 3.2× with 4, then stays flat at 2.4 to 3.0× from 5 to 8 threads.

1. **The power limit lowers the clock as cores are added** (4.12 GHz with 1 busy thread, 3.43 with 4, 2.80 with 8). The kernel is compute bound, so even perfect parallel code can only reach

(cores used) × (clock with p threads) / (clock with 1 thread):

2 threads: $2 \times \frac{3.97}{4.12} = 1.93\times$ 4 threads: $4 \times \frac{3.43}{4.12} = 3.33\times$ 8 threads: $4 \times \frac{2.80}{4.12} = 2.72\times$

In the best case `attn_omp` reaches 1.89× (2 threads), 3.18× (4 threads) and 2.46 to 2.78× (8 threads): about 90 to 100% of these ceilings. Most of the "lost" efficiency is lost clock frequency, not parallel overhead.

2. **There are only 4 physical cores.** Threads 5 to 8 run on the second hyperthread of a core, and a core has one 512 bit FMA unit whether one or two threads use it. So the time stays flat beyond 4 threads.
3. **WSL2 adds noise.** The Windows hypervisor sometimes places two busy vCPUs on the same physical core (the 4 thread FMA test got only 22.6 FLOP/cycle per core), which explains the irregular 4 to 6 thread results. The same noise made the 1 thread run at $N = 4096$, $d = 64$ 12% slower than `attn_best` (56.3 against 50.4 ms), which is why its 7 thread speedup (3.01×) is above the ceiling. Measured against `attn_best`, the best speedups are 3.32×, 2.84×, 2.69× and 2.64×, all within the ceilings.
4. **Memory is not the limit:** the DRAM intensity is 1024 FLOP/B and each thread reuses its K and V blocks 32 times from L1. $d = 128$ scales less well at 3 to 4 threads, likely because each thread streams 4 MB of K and V from the shared 8 MB L3; the noise is too large to confirm this.

Q5. Roofline

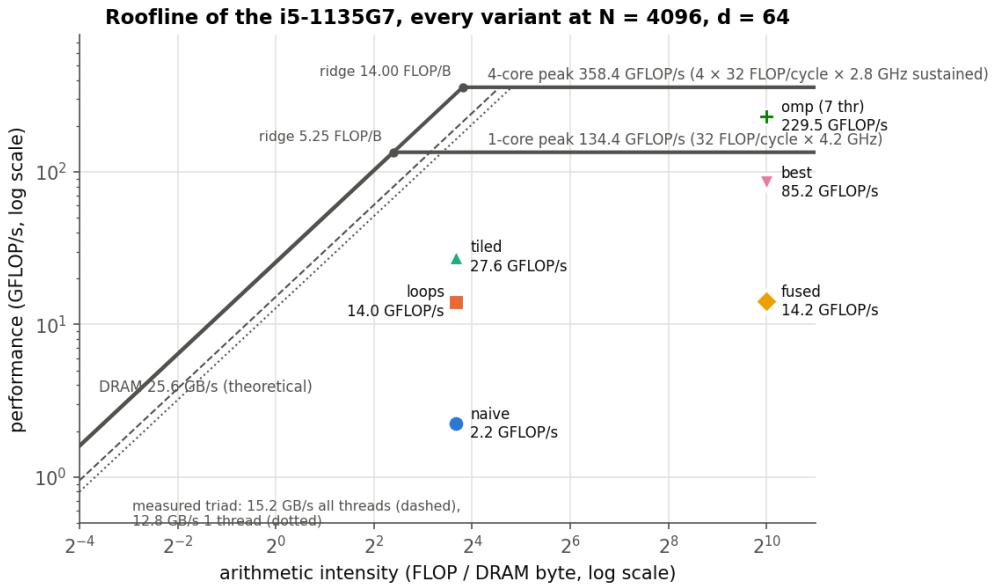


Figure 4: Roofline from the Q1 numbers (134.4 and 358.4 GFLOP/s, 25.6 GB/s, ridges at 5.25 and 14.0 FLOP/B), with every variant at $N = 4096$, $d = 64$. Dashed and dotted lines: measured triad bandwidth. `omp` is shown at its fastest thread count.

How the arithmetic intensity was computed. AI = FLOPs/DRAM bytes, with FLOPs $= 4N^2d$. Hardware counters are not available in WSL2, so the DRAM traffic was counted from the code (FP32, 4 B per element):

- **Every variant** reads Q, K, V and writes O: $16Nd$ bytes (4.2 MB). The repeated reads of K and V in `naive`, `loops` and `fused` hit in L2 or L3, not DRAM.
- **`naive`, `loops`, `tiled`** also move the 64 MB S through DRAM: written once ($8N^2$ with the write allocate read), read and written back by softmax ($8N^2$), and read by SV ($4N^2$), a total of $20N^2$ bytes (335.5 MB).

$$AI_{\text{naive, loops, tiled}} = \frac{4N^2d}{16Nd + 20N^2} = \frac{4.29 \text{ GFLOP}}{339.7 \text{ MB}} = 12.6 \ \frac{\text{FLOP}}{\text{B}}$$

$$AI_{\text{fused, best, omp}} = \frac{4N^2d}{16Nd} = \frac{N}{4} = 1024 \ \frac{\text{FLOP}}{\text{B}}$$

variant	AI (FLOP/B)	GFLOP/s	roof at this AI	% of roof
naive	12.6	2.2	134.4 (1 core)	2%
loops	12.6	14.0	134.4 (1 core)	10%
tiled	12.6	27.6	134.4 (1 core)	21%
fused	1024	14.2	134.4 (1 core)	11%
best	1024	85.2	134.4 (1 core)	63%
omp (7 threads)	1024	229.5	358.4 (all cores)	64%

Table 7: Every variant at $N = 4096$, $d = 64$ against its roof.

What limits each variant. Every variant is to the right of its ridge point, so none is limited by DRAM bandwidth. The limits are all inside the core or the caches:

- **naive:** the latency of dependent scalar FMAs and the column walk of V (Q2).
- **loops:** L2 bandwidth, since there is no register reuse, plus the scalar softmax (Q3a).
- **tiled:** the scalar softmax (56% of the time) and the write allocate misses on the stored S (Q3b).
- **fused:** far into the compute bound region, yet at only 11% of the roof, because the limit is between the caches and the core. It reads K and V again for every row, so its intensity against L2 and L3 traffic is only 0.5 FLOP/B, which allows 11 to 28 GFLOP/s (Q3c).
- **best:** blocking raises the intensity against cache traffic to 16 FLOP/B, so even L3 (22 GB/s) could feed 352 GFLOP/s. It is compute bound at every level, and what remains is the FMA unit itself, shared by the matmuls, the exp polynomial and the reductions (Q3d).
- **omp:** compute bound like `best`; limited by the lower all core clock, the 4 physical cores and WSL2's vCPU placement (Q4).